Week 5: Early Design and Proof-of-Concept Planning

Temporal CNN for DDoS Attack Detection

Lokesh L K S (ljl5837@psu.edu)

The Pennsylvania State University

8th February 2026

1. Goal

Translate the foundational work completed so far (dataset preprocessing, baseline CNN) into an initial proof-of-concept design while continuing development of the temporal image pipeline. This week focuses on validating the temporal CNN approach for DDoS attack detection and addressing technical barriers before baseline benchmarking.

2. Week 4 Completion Status

Before proceeding with Week 5 activities, the following Phase 1 deliverables have been completed:

Component	Details	Status
Dataset Preprocessing	BCCC-cPacket-Cloud-DDoS-2024 cleaned, normalized [0-255], SMOTE balanced, stratified splits (70/15/15)	**Complete**
Baseline CNN	2-layer Conv2D (32/64 filters), MaxPooling, Dense(128), Dropout(0.5), sigmoid output	**Complete**
Temporal Image Pipeline	Flow-to-image conversion with time window testing (5s, 10s, 20s, 50s)	**In Progress**

3. Activities

3.1 Hands-on Framework Integration in Security Context

Practical application of AI frameworks to the DDoS detection problem, building on the baseline CNN already implemented in TensorFlow 2.x with Keras API.

Temporal CNN Integration Testing

- Continue development of temporal image pipeline: finalize flow-to-image conversion logic and test time window sizes (5s, 10s, 20s, 50s)
- Validate partial end-to-end data flow: raw BCCC flows → temporal images → CNN input format verification

- Run initial inference tests on sample batches once pipeline produces valid temporal images
- Design integration plan for connecting completed temporal pipeline to baseline CNN architecture

Framework-Specific Exploration

- Evaluate TensorFlow profiling tools (tf.profiler) for identifying computational bottlenecks in temporal image processing
- Test TensorFlow data pipeline optimizations (tf.data.Dataset) for efficient temporal image batch loading
- Explore model serialization formats for potential deployment scenarios (SavedModel, TFLite)
- Document framework-specific considerations for the cybersecurity deployment context

3.2 Technical Barrier Analysis and Resource Assessment

Systematic identification and mitigation planning for technical challenges ahead of the optimization phase (Weeks 8-11).

Identified Technical Barriers

Barrier	Impact	Mitigation Strategy
Temporal image size variability	Inconsistent input dimensions across time windows may cause training instability	Implement padding/truncation to fixed dimensions; test multiple window sizes
Class imbalance in multi-class	17 attack types have uneven representation despite SMOTE on binary labels	Evaluate per-class SMOTE ratios; consider focal loss for multi-class extension
Computational cost	Temporal image generation creates larger inputs than raw flow features	Profile memory/compute on Paperspace; implement batch generators; consider 1D convolutions as fallback
Adversarial robustness	Crafted flows could evade temporal pattern detection	Plan adversarial evaluation during Week 12-13; implement input validation checks

Resource Requirements

- Compute: Paperspace GPU instances for training; Penn State cluster access as backup
- Storage: Estimated 5-10 GB for temporal image dataset depending on final window size
- Time: Temporal image generation estimated at 2-4 hours for full dataset on Paperspace GPU
- Software: TensorFlow 2.x, NumPy, scikit-learn, matplotlib for visualization

4. Milestone / Deliverable: Draft Prototype Design

4.1 Initial Code Base Structure

The project follows a modular architecture enabling independent iteration on each component. Current directory structure:

```
ddos-temporal-cnn/
├── data/
│   ├── raw/                # BCCC-cPacket-Cloud-DDoS-2024 dataset
│   ├── processed/          # Cleaned, normalized flow data
│   └── temporal_images/    # Generated temporal image arrays
├── src/
│   ├── preprocessing/
│   │   ├── data_cleaner.py       # Normalization, feature selection
│   │   ├── temporal_converter.py  # Flow-to-temporal-image pipeline
│   │   └── smote_balancer.py     # Class balancing with SMOTE
│   ├── models/
│   │   ├── baseline_cnn.py       # Baseline CNN architecture
│   │   ├── temporal_cnn.py       # Temporal CNN (planned)
│   │   └── model_utils.py        # Training, callbacks, checkpointing
│   └── evaluation/
│       ├── metrics.py           # Accuracy, Precision, Recall, F1
│       └── visualizations.py     # Confusion matrices, ROC curves
├── notebooks/
│   ├── 01_data_exploration.ipynb
│   ├── 02_baseline_training.ipynb
│   └── 03_temporal_pipeline.ipynb
├── configs/
│   └── hyperparams.yaml      # Centralized hyperparameter config
└── results/
    ├── models/             # Saved model checkpoints
    └── logs/               # Training logs, TensorBoard
```

4.2 Conceptual System Diagram

The proof-of-concept integrates four pipeline stages, each independently testable:

End-to-End Pipeline Architecture

[Stage 1: Data Ingestion] → [Stage 2: Temporal Imaging] → [Stage 3: CNN Model] → [Stage 4: Evaluation]

BCCC Dataset → Clean & Normalize → Time-Window Grouping → 2D Image Arrays → Conv2D Layers → Classification → Metrics

4.3 How AI Integrates with the Cybersecurity Problem

The core innovation of this project is converting temporal network flow sequences into 2D image representations that CNNs can process for DDoS detection. This approach bridges computer vision and network security:

The Temporal Image Approach

Problem: Traditional DDoS detection relies on threshold-based rules or statistical methods that struggle with evolving attack patterns. Raw network flow features are high-dimensional (300+ features in BCCC dataset) and sequential, making direct classification challenging.

Solution: By grouping consecutive network flows within configurable time windows and arranging their normalized features into 2D arrays, we create "temporal images" that encode both spatial relationships between flow features and temporal evolution of traffic patterns. CNNs excel at detecting spatial patterns in image data, enabling them to learn attack signatures that manifest as visual patterns in these temporal representations.

Security Value: This approach can detect DDoS attacks across 17 different attack types in the BCCC-cPacket-Cloud-DDoS-2024 dataset. The lightweight model target (<5,000 parameters) makes it feasible for deployment in resource-constrained network monitoring environments. The temporal dimension captures attack evolution patterns that single-flow analysis would miss.

Key Design Decisions

Decision	Rationale	Alternative Considered
CNN over LSTM	CNNs process temporal images efficiently with fewer parameters; spatial pattern detection aligns with image representation	*LSTM for raw sequence processing (planned as optional Week 14 comparison)*
Temporal images vs. raw features	2D representation captures both inter-feature and temporal correlations simultaneously	*Direct feature vector input to Dense layers (loses temporal context)*
Binary first, multi-class later	Establishes strong baseline before tackling 17-class problem; reduces initial complexity	*Multi-class from start (higher risk of poor initial results)*
[0-255] normalization	Maps to standard image pixel range; enables use of proven image processing techniques	*Min-max to [0-1] (less intuitive for image-based interpretation)*
SMOTE balancing	Addresses class imbalance between benign and attack samples in training data	*Weighted loss function (doesn't generate synthetic training examples)*

5. Next Steps (Weeks 6-7: Baseline Benchmarking)

With the proof-of-concept design established and temporal pipeline nearing completion, Weeks 6-7 will focus on establishing quantitative baseline metrics:

1. Run full temporal CNN training on complete dataset with best-performing time window size
2. Establish baseline performance metrics: Accuracy, Precision, Recall, F1-Score across all test splits
3. Generate confusion matrices to identify which attack types are most/least detectable
4. Profile resource usage: memory footprint, training time per epoch, inference latency per sample
5. Document parameter counts and computational complexity for optimization targeting in Weeks 8-11